

1 Global vs. Local Interpretability

**Global SHAP importance (mean	SHAP
BotAttack	1. Port 2. Payload_Size 3. Status
Normal	1. Port 2. Payload_Size 3. Status
PortScan	1. Payload_Size 2. Port 3. Status

What this tells us – Across all three outcomes the tree ensemble relies most on *the encoded “Port” value*, followed by *the magnitude of the payload* and finally the *success/failure flag*. The three attack subclasses therefore diverge mainly in the relative weight given to *Port* versus *Payload_Size*; PortScan’s decisions are driven almost entirely by the payload.

The LIME rule-sets confirm these global signals on a per-instance basis:

- **Port** is the decisive factor.
 - Instances with `1 < Port < 4` receive a large positive weight, while anything larger than 5 brings a strong negative contribution.
 - The rare case of `Port=0` (mapped to 21) also receives a positive weight, hinting that low numbers sometimes signal benign traffic.
- **Payload_Size** is another key predictor.
 - Lower values (i.e., negative after scaling) shift the decision toward “Normal”, whereas payloads close to zero push the prediction toward an attack.
- **Status (Success)** remains a small yet consistent positive weight in all LIME rules, reflecting the general expectation that successful requests are more likely legitimate.

The alignment of SHAP and LIME on these three variables underscores **model reliability** – both at the level of a forest average (SHAP) and at the level of instance-specific rule activation (LIME).

2 Feature Influence on Attack vs. Normal

1. Port (encoded)

- **In-depth** – The encoder maps port numbers to discreet integer codes, but the tree model treats these codes as **ordinal**.
- *Low codes* (1–4) → positive influence for Normal.
- *High codes* (5 and above) → negative influence for Attack, mirroring real-world behaviour: many scanners use high ports (e.g., 4444, 8080, 31337).

2. **Payload_Size**
 - Negative (after standard scaling) → strong indicator of normal traffic.
 - Near-zero or slightly positive payloads → likely a scan or malicious request, as they indicate minimal data transfer typical of probes.
3. **Status**
 - Success (1) tends to reinforce “Normal” predictions, though its weight is smaller than the other two features.
4. **Other Features (Request_Type, Protocol, User_Agent)**
 - Both SHAP mean values and LIME rule contributions for these variables are comparatively low.
 - The RandomForest likely models interactions that involve them, but they don’t drive the final outcome on their own.

3 Patterns & Correlations in the Dataset

Observed Pattern	Interpretation
Low port numbers + negative payload → Normal	Follows the intuition that legitimate services (HTTP at 80, SSH at 22, etc.) usually respond with a modest payload, and the encoded port values capture this.
High port numbers + payload close to 0 → Scan_Attack or PortScan	Signature of scanners: they probe multiple ports with minimalist packets to discover open services.
Status is consistently successful for “Normal”	Successful requests tend to be legitimate; a few malicious requests also succeed, but the forest relies more on port/payload.
Categorical encodings are ordinal rather than one-hot	This can create spurious linear ordering (e.g., port code 5 < code 6 < code 7). While RandomForest can handle this, it may push the model to consider port order as predictive, amplifying the port effect.

4 Analysis of Prediction Samples & Misclassifications

All 15 prediction pairs had `true_label=1` and `predicted_label=1`.

- **Confidence** – The model is consistently correct on the “Normal” class for the sampled logs.
- **Boundary Cases** – LIME contributions for the first few instances show negative weights for high ports, yet the predictions remain Normal because Port was in the low-to-mid range ($1 < \text{Port} \leq 4$).
- **Model Sensitivity** – Had we seen a sample with an out-of-range port (e.g., > 10) and a small payload, the forest would likely switch to an attack label with high certainty.

Conclusion: In this little test set, the model shows perfect precision on Normal, but the sample is too small to gauge recall. With a larger variety of edge cases (e.g., port 4444 with large payload), we might observe false positives.

5 Impact of Categorical Encodings

- **Ordinal Encoding for Categorical Fields** – Request_Type, Protocol, User_Agent, and even Port are treated as ordered integers.
 - For tree-based models this is *acceptable*, but the implicit ordering introduces bias.
 - Example: A path from a leaf that checks $\text{Port} \leq 5$ later steps toward $\text{Port} > 5$. The model implicitly learns that “later” ports are less likely to be normal.
- **Potential Misleading Signals** – If two distinct categorical values happen to be close numerically (e.g., Port 21→0 and Port 22→1), the forest might regard them as similar even though their semantics differ.
- **Recommendation** – For future runs, consider a **One-Hot Encoder** or **Target-Mean Encoding** for features like User_Agent or Request_Type, and a **custom order** for Port (e.g., mapping all well-known service ports to a separate “service-port” bucket).

6 Cybersecurity Insights

1. **Port-Centric Scanning Behaviour** – Attack logs most often cluster around high port codes, with payload sizes remaining minuscule.
 - *Mitigation:* Harden high-port services, implement rate limiting, or deploy honeypots on those ports to detect scanning.
2. **Payload Size as a Grey-Area Weapon** – While many scans keep payloads at zero, some attack samples show slightly larger payloads.

- *Defense*: Monitors that flag unusual spikes in payload even when port is benign can flag “slow-loris” or “steg induced” techniques.
3. **Status Reflection** – The small positive contribution for success suggests that attackers hit a service that is authenticated or threshold-checked before revealing more data.
 - *Countermeasure*: Require stricter authentication and logging on Successful / Redirect responses.
 4. **User_Agent & Protocol Influence is Subtle** – They are not primary discriminators, but may help in edge-cases (e.g., an attacker using `Wget` or `Nmap` over ICMP).
 - *Opportunity*: Enrich threat intelligence by correlating User_Agent patterns with known reconnaissance tools.

7 Suggested Improvements

Area	Action
Feature Engineering	• Add explicit “known service port” flag (binary). • Encode payload as <i>log-scale</i> or <i>count of non-ASCII bytes</i> to capture tunnelling.
Encoding	• One-Hot encode User_Agent & Request_Type; treat Port as a categorical bucket (e.g., Service_Port / Well_Known / Unspecified).
Model Tuning	• Retune RandomForest hyper-parameters (e.g., <code>max_features</code>, <code>min_samples_leaf</code>) to reduce over-fitting on the highly ordered Port column.
Dataset Augmentation	• Introduce synthetic scans on mid-range ports (≤ 4) to see if the model overshadows them. • Inject labelled “mixed-type” attacks (e.g., port scan with larger payload).
Explainability Consistency	• Validate SHAP vs. LIME alignment on a broader sample set to ensure no local-global discrepancy.
Operational Deployment	• Build real-time anomaly detection that combines the model’s probability scores with statistical thresholds on port usage counts per minute.

Area	Action
Security Focus	Correlate flagged attack instances with external threat feeds (CVE, IOC) to confirm legitimate versus false-positive scans.

Closing Summary

The RandomForest model is largely driven by **Port** and **Payload_Size** – a finding that aligns with common scanning behaviours. SHAP and LIME agree on this, with ports >5 and near-zero payloads tipping the decision toward an attack. While categorical encodings provide a quick way to feed the model, they risk injecting artificial order; adjusting these will likely sharpen the model’s precision and reduce unintended bias. Finally, integrating these insights into a layered security workflow (rate-limiting, honeypots, and dynamic whitelisting) will bolster defense against both the plaintext and subtle attack vectors this dataset reveals.